Django Introduction (Detailed Guide for PDF)

What is Django?

Django is a high-level Python web framework used to build secure, scalable, and fast web applications.

History of Django

- Created by Adrian Holovaty and Simon Willison
- Released in 2005
- Maintained by the Django Software Foundation

Why Use Django?

- Rapid development
- Built-in security features
- Scalable and robust
- Follows DRY (Don't Repeat Yourself) principle

Features of Django

- MTV architecture (Model-Template-View)
- Built-in admin panel
- ORM (Object Relational Mapper)
- Authentication system
- URL routing

Django Architecture (MTV)

- Model → Handles data and database
- Template → Handles UI (HTML)
- View → Handles logic and request/response

Installing Django

```
pip install django
```

Creating a Project

```
django-admin startproject myproject
cd myproject
python manage.py runserver
```

Creating an App

```
python manage.py startapp myapp
```

Project Structure

- manage.py
- settings.py
- urls.py
- apps folder

Models

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

Migrations

```
python manage.py makemigrations
python manage.py migrate
```

Views

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Hello Django")
```

URLs

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home),
]
```

Templates

```
<h1>Hello {{ name }}</h1>
```

Admin Panel

```
python manage.py createsuperuser
```

Access: http://127.0.0.1:8000/admin

Forms

```
from django import forms

class MyForm(forms.Form):
    name = forms.CharField()
```

Static Files

Used for CSS, JS, images.

Django ORM

```
Student.objects.all()
```

```
Student.objects.filter(age=20)
```

Example Project

- Create model
- Add view
- Connect URL
- Display data in template

Advantages of Django

- Fast development
- Secure
- Scalable

Disadvantages of Django

- Heavy for small projects
- Learning curve for beginners

Best Practices

- Follow project structure
- Use reusable apps
- Keep code modular